

LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 4



Disusun Oleh:

Muhammad Althaf Mulya (2511533018)

Asisten Praktikum:

Sherly Sukmadira

Dosen Pengampu:

Wahyudi Dr. S.T, M.T

**DEPARTEMEN INFORMATIKA FAKULTAS
TEKNOLOGI INFORMASI UNIVERSITAS
ANDALAS**

2025

Daftar Isi

DAFTAR ISI	2
KATA PENGANTAR	4
1.1 Latar Belakang	5
2.1 Class NodeDLL	6
2.2 Program InsertDLL	7
2.3 Program PenelusuranDLL	8
2.4 Program HapusDLL	9
3.1 Ringkasan Hasil Praktikum	18

Kata Pengantar

Puji syukur kehadiran Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum dengan judul “Double Linked List” ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep DLL dalam bahasa pemrograman Java.

BAB I

PENDAHULUAN

Puji syukur kehadiran Allah SWT atas segala limpahan rahmat, taufik, dan hidayah-Nya, sehingga penulis dapat menyelesaikan laporan praktikum Struktur Data Pekan 6 dengan topik "Double Linked List" ini tepat pada waktunya.

Laporan ini disusun untuk memenuhi tugas praktikum mata kuliah Struktur Data sekaligus sebagai dokumentasi atas pemahaman mengenai konsep linear linked list yang memiliki dua arah penelusuran. Melalui praktikum ini, penulis telah mempelajari dan mengimplementasikan berbagai operasi pada *Double Linked List*, mulai dari operasi penyisipan (*insertion*), penelusuran (*traversal*), penghapusan (*deletion*), hingga studi kasus nyata berupa sistem antrean lagu (*playlist*).

Dalam penyusunan laporan ini, penulis menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Wahyudi, S.T., M.T., selaku Dosen Pengampu mata kuliah Struktur Data yang telah memberikan bimbingan teori dan ilmu yang bermanfaat.
2. Kak Sherly Sukmadira, selaku Asisten Praktikum yang telah memberikan arahan, asistensi, dan masukan teknis selama proses praktikum di laboratorium.
3. Rekan-rekan mahasiswa Departemen Informatika angkatan 2025 yang telah saling mendukung dalam proses pembelajaran.

Penulis menyadari bahwa dalam laporan ini masih terdapat kekurangan, baik dari segi teknis penulisan maupun kedalaman materi. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi penulis khususnya, dan bagi pembaca pada umumnya dalam memahami implementasi struktur data di bahasa pemrograman Java.

BAB II

PEMBAHASAN

2.1 Class Node DLL

Adalah sebuah class yang terdiri atas 3 atribut dan 1 constructor, yaitu atribut data, atribut next, dan prev yang masing-masing berguna sebagai menyimpan data, dan duanya untuk pointer ke next atau prev node.

```
NodeDLL_2511533018.java X
package pekan6_2511533018;

public class NodeDLL_2511533018
{
    int data_3018; // data
    NodeDLL_2511533018 next_3018; // pointer ke next node
    NodeDLL_2511533018 prev_3018; // pointer ke previous node

    // Constructor
    public NodeDLL_2511533018(int data)
    {
        this.data_3018 = data;
        this.next_3018 = null;
        this.prev_3018 = null;
    }
}
```

2.2 Program Insert DLL

Adalah sebuah class untuk memasukan atau membuat node baru, terdiri atas 5 methods, yaitu insertBegin untuk isi node dari depan, insertEnd untuk isi node dari belakang, insertAtPosition untuk insert node pada posisi tertentu, printList untuk menampilkan output dan main sebagai entry point.

```

public class InsertDLL_2511533018
{
    public static NodeDLL_2511533018 insertBegin_2511533018(
        NodeDLL_2511533018 head_3018,
        int data)
    {
        NodeDLL_2511533018 new_node_3018 = new NodeDLL_2511533018(data);

        new_node_3018.next_3018 = head_3018;

        if (head_3018 != null)
        {
            head_3018.prev_3018 = new_node_3018;
        }

        return new_node_3018;
    }

    public static NodeDLL_2511533018 insertEnd_2511533018(
        NodeDLL_2511533018 head_3018,
        int newData)
    {
        NodeDLL_2511533018 new_node_3018 = new NodeDLL_2511533018(newData);

        if (head_3018 == null)
        {
            head_3018 = new_node_3018;
        }
        else
        {
            NodeDLL_2511533018 curr_3018 = head_3018;
            while (curr_3018.next_3018 != null)
            {
                curr_3018 = curr_3018.next_3018;
            }
            curr_3018.next_3018 = new_node_3018;
            new_node_3018.prev_3018 = curr_3018;
        }
        return head_3018;
    }

    public static NodeDLL_2511533018 insertAtPosition(
        NodeDLL_2511533018 head_3018,
        int pos,
        int new_data)
    {
        NodeDLL_2511533018 new_node_3018 = new NodeDLL_2511533018(new_data);

        if (pos == 1)
        {
            new_node_3018.next_3018 = head_3018;
            if (head_3018 != null)
            {
                head_3018.prev_3018 = new_node_3018;
            }
            head_3018 = new_node_3018;
            return head_3018;
        }

        NodeDLL_2511533018 curr_3018 = head_3018;

        for (int i = 1; i < pos - 1 && curr_3018 != null; i++)
        {
            curr_3018 = curr_3018.next_3018;
        }

        if (curr_3018 == null)
        {
            System.out.println("Posisi tidak ada.");
            return head_3018;
        }
    }
}

```

```

        new_node_3018 .prev_3018 = curr_3018;
        new_node_3018.next_3018 = curr_3018.next_3018;
        curr_3018.next_3018 = new_node_3018;

        if (new_node_3018.next_3018 != null)
        {
            new_node_3018.next_3018.prev_3018 = new_node_3018;
        }

        return head_3018;
    }

    public static void printList_2511533018(NodeDLL_2511533018 head_3018)
    {
        NodeDLL_2511533018 curr_3018 = head_3018;

        while (curr_3018 != null)
        {
            System.out.print(curr_3018.data_3018 + " <-> ");
            curr_3018 = curr_3018.next_3018;
        }

        System.out.println();
    }

    public static void main(String[] args) {
        NodeDLL_2511533018 head_3018_3018 = new NodeDLL_2511533018(2);
        head_3018_3018.next_3018 = new NodeDLL_2511533018(3);
        head_3018_3018.next_3018.prev_3018 = head_3018_3018;
        head_3018_3018.next_3018.next_3018 = new NodeDLL_2511533018(5);
        head_3018_3018.next_3018.next_3018.prev_3018 = head_3018_3018;

        System.out.print("DLL Awal: ");
        printList_2511533018(head_3018_3018);

        head_3018_3018 = insertBegin_2511533018(head_3018_3018, 1);
        System.out.print("Simpul 1 ditambah di awal: ");
        printList_2511533018(head_3018_3018);

        System.out.print("Simpul 6 ditambah di akhir: ");
        int data_3018 = 6;
        head_3018_3018 = insertEnd_2511533018(head_3018_3018, data_3018);
        printList_2511533018(head_3018_3018);

        System.out.print("Tambah node 4 di posisi 4: ");
        int data2_3018 = 4;
        int pos_3018 = 4;
        head_3018_3018 = insertAtPosition(head_3018_3018, pos_3018, data2_3018);
        printList_2511533018(head_3018_3018);
    }
}

```

Output:

```

C:\Users\user> insertDLL_2511533018 [Java Application] /Library/Java/javavirtuallmachines/jdk-20-jdk/Contents/Home/bin/java
DLL Awal: 2 <-> 3 <-> 5 <->
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

2.3 Program Penelusuran DLL

Adalah sebuah class untuk melakukan penelusuran pada setiap node, terdiri atas 3 methods, yaitu forwardTraversal yang berguna untuk melakukan penelusuran dari depan, backwardTraversal untuk penelusuran dari belakang dan main sebagai entry point.

```
package pekan6_2511533018;

public class PenelusuranDLL_2511533018
{
    public static void forwardTraversal_2511533018(NodeDLL_2511533018 head_3018)
    {
        NodeDLL_2511533018 curr_3018 = head_3018;

        while (curr_3018 != null)
        {
            System.out.print(curr_3018.data_3018 + " <-> ");
            curr_3018 = curr_3018.next_3018;
        }

        System.out.println();
    }

    public static void backwardTraversal(NodeDLL_2511533018 tail)
    {
        NodeDLL_2511533018 curr_3018 = tail;
        while (curr_3018 != null)
        {
            System.out.print(curr_3018.data_3018 + " <-> ");
            curr_3018 = curr_3018.prev_3018 ;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeDLL_2511533018 head_3018_3018 = new NodeDLL_2511533018(1);
        NodeDLL_2511533018 second_3018 = new NodeDLL_2511533018(2);
        NodeDLL_2511533018 third_3018 = new NodeDLL_2511533018(3);

        head_3018_3018.next_3018 = second_3018;
        second_3018.prev_3018 = head_3018_3018;
        second_3018.next_3018 = third_3018;
        third_3018.prev_3018 = second_3018;

        System.out.println("Penelusuran maju: ");
        forwardTraversal_2511533018(head_3018_3018);

        System.out.println("Penelusuran mundur: ");
        backwardTraversal(third_3018);
    }
}
```

Output:

```
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

2.4 Program Hapus DLL

Adalah sebuah class untuk melakukan penghapusan pada suatu node, terdiri atas 5 methods, yaitu deleteHead untuk menghapus dari depan, deleteLast untuk menghapus dari belakang, deletePos utk delete node berdasarkan psosi tertentu, printList untuk melihat outputnya, dan main sebagai entry point.

```
package pekan6_2511533018;

public class HapusDLL_2511533018 {

    public static NodeDLL_2511533018 deleteHead_2511533018(NodeDLL_2511533018 head_3018)
    {
        if (head_3018 == null)
        {
            return null;
        }

        head_3018 = head_3018.next_3018;

        if (head_3018 != null)
        {
            head_3018.prev_3018 = null;
        }

        return head_3018;
    }

    public static NodeDLL_2511533018 deleteLast_2511533018(NodeDLL_2511533018 head_3018)
    {
        if (head_3018 == null)
        {
            return null;
        }

        if (head_3018.next_3018 == null)
        {
            return null;
        }

        NodeDLL_2511533018 curr_3018 = head_3018;
        while (curr_3018.next_3018 != null)
        {
            curr_3018 = curr_3018.next_3018;
        }

        if (curr_3018.prev_3018 != null)
        {
            curr_3018.prev_3018.next_3018 = null;
        }

        return head_3018;
    }
}
```

```

public static NodeDLL_2511533018 deletePos_2511533018(NodeDLL_2511533018 head_3018, int pos_3018)
{
    if (head_3018 == null)
    {
        return head_3018;
    }

    NodeDLL_2511533018 curr_3018 = head_3018;

    // 1 -> i
    for (int i = 1; curr_3018 != null && i < pos_3018; i++)
    {
        curr_3018 = curr_3018.next_3018;
    }

    if (curr_3018 == null)
    {
        return head_3018;
    }

    if (curr_3018.prev_3018 != null)
    {
        curr_3018.prev_3018.next_3018 = curr_3018.next_3018;
    }

    if (curr_3018.next_3018 != null)
    {
        curr_3018.next_3018.prev_3018 = head_3018;
    }

    if (head_3018 == curr_3018)
    {
        head_3018 = curr_3018.next_3018;
    }

    return head_3018;
}

public static void printList_2511533018(NodeDLL_2511533018 head_3018)
{
    NodeDLL_2511533018 curr_3018 = head_3018;

    while (curr_3018 != null)
    {
        System.out.print(curr_3018.data_3018 + " <-> ");
        curr_3018 = curr_3018.next_3018;
    }

    System.out.println();
}

public static void main(String[] args) {
    NodeDLL_2511533018 head_3018 = new NodeDLL_2511533018(1);
    head_3018.next_3018 = new NodeDLL_2511533018(2);
    head_3018.next_3018.prev_3018 = head_3018;
    head_3018.next_3018.next_3018 = new NodeDLL_2511533018(3);

    head_3018.next_3018.next_3018.prev_3018 = head_3018.next_3018;
    head_3018.next_3018.next_3018.next_3018 = new NodeDLL_2511533018(4);
    head_3018.next_3018.next_3018.next_3018.prev_3018 = head_3018.next_3018.next_3018;
    head_3018.next_3018.next_3018.next_3018.next_3018 = new NodeDLL_2511533018(5);
    head_3018.next_3018.next_3018.next_3018.next_3018.prev_3018 = head_3018.next_3018.next_3018.next_3018;

    System.out.print("DLL Awal: ");
    printList_2511533018(head_3018);

    System.out.print("Setelah head dihapus: ");
    head_3018 = deleteHead_2511533018(head_3018);
    printList_2511533018(head_3018);

    System.out.print("Setelah node terakhir dihapus: ");
    head_3018 = deleteLast_2511533018(head_3018);
    printList_2511533018(head_3018);

    System.out.print("Menghapus node ke 2: ");
    head_3018 = deletePos_2511533018(head_3018, 2);
    printList_2511533018(head_3018);
}

```

Output:

```
DLL Awal: 1 <--> 2 <--> 3 <--> 4 <--> 5 <-->  
Setelah head dihapus: 2 <--> 3 <--> 4 <--> 5 <-->  
Setelah node terakhir dihapus: 2 <--> 3 <--> 4 <-->  
Menghapus node ke 2: 2 <--> 4 <-->
```

2.5 Tugas Pekan 6

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah

Pembahasan:

Kita buat class Lagu yang berisikan atribut judul, penyanyi sebagai metadata, lalu next dan prev sebagai pointer, kemudian 1 constructor dan masing-masing setter getter.

```

package pekan6_2511533018;

public class Lagu_2511533018
{
    private String judul_3018;
    private String penyanyi_3018;
    Lagu_2511533018 next_3018;
    Lagu_2511533018 prev_3018;

    // Constructor
    public Lagu_2511533018(String judul, String penyanyi)
    {
        this.judul_3018 = judul;
        this.penyanyi_3018 = penyanyi;
        this.next_3018 = null;
        this.prev_3018 = null;
    }

    // Getter dan Setter
    public String getJudul_3018()
    {
        return judul_3018;
    }

    public void setJudul_3018(String judul)
    {
        this.judul_3018 = judul;
    }

    public String getPenyanyi_3018()
    {
        return penyanyi_3018;
    }

    public void setPenyanyi_3018(String penyanyi)
    {
        this.penyanyi_3018 = penyanyi;
    }
}

```

Lalu kita punya sebuah class untuk implemnatasi Musik yang berisikan 6 method dan 2 atribut yaitu head dan tail, lalu ada method

```

import java.util.Scanner;

public class Musik_2511533018
{
    private Lagu_2511533018 head_3018;
    private Lagu_2511533018 tail_3018;

    public void tambahLagu_3018(String judul, String penyanyi)
    {
        Lagu_2511533018 laguBaru = new Lagu_2511533018(judul, penyanyi);

        if (head_3018 == null)
        {
            head_3018 = tail_3018 = laguBaru;
        }
        else
        {
            tail_3018.next_3018 = laguBaru;
            laguBaru.prev_3018 = tail_3018;
            tail_3018 = laguBaru;
        }

        System.out.println("Lagu '" + judul + "' berhasil ditambahkan!");
    }
}

```

Yang akan berguna untuk menambahkan lagu, method menginisiasi ADT Lagu, lalu melakukan validasi dan create kalau cocok.

```

}

public void hapusLaguAwal_3018()
{
    if (head_3018 == null)
    {
        System.out.println("Playlist kosong, gagal menghapus.");
        return;
    }

    System.out.println("Menghapus lagu: " + head_3018.getJudul_3018());

    if (head_3018 == tail_3018)
    {
        head_3018 = tail_3018 = null;
    }
    else
    {
        head_3018 = head_3018.next_3018;
        head_3018.prev_3018 = null;
    }
}

```

Method untuk menghapus lagu di posisi awal dengan validasi is null dan validasi is head adalah tail, jika iya maka masukan tail ke dalam head kalau tidak create head baru dengan nilai head milik si ADT musik.

```

public void tampilMaju_3018()
{
    if (head_3018 == null)
    {
        System.out.println("Playlist Kosong.");
        return;
    }

    Lagu_2511533018 temp = head_3018;
    System.out.println("\n--- Playlist (Maju) ---");

    while (temp != null)
    {
        System.out.println("- " + temp.getJudul_3018() + " (" + temp.getPenyanyi_3018() + ")");
        temp = temp.next_3018;
    }
}

```

Method untuk menampilkan lagu dari urutan awal.

```

public void tampilMundur_3018()
{
    if (tail_3018 == null)
    {
        System.out.println("Playlist Kosong.");
        return;
    }

    Lagu_2511533018 temp = tail_3018;
    System.out.println("\n--- Playlist (Mundur) ---");

    while (temp != null)
    {
        System.out.println("- " + temp.getJudul_3018() + " (" + temp.getPenyanyi_3018() + ")");
        temp = temp.prev_3018;
    }
}

```

Method untuk menampilkan lagu dari belakang.

```

,
public void cariLagu_3018(String judul_3018)
{
    Lagu_2511533018 temp_3018 = head_3018;
    boolean ditemukan = false;

    while (temp_3018 != null)
    {
        if (temp_3018.getJudul_3018().equalsIgnoreCase(judul_3018))
        {
            System.out.println("Lagu Ditemukan: " + temp_3018.getJudul_3018() + " oleh " + temp_3018.getPenya
            ditemukan = true;
            break;
        }

        temp_3018 = temp_3018.next_3018;
    }

    if (!ditemukan)
        System.out.println("Lagu dengan judul '" + judul_3018 + "' tidak ada di playlist.");
}

```

Method untuk mencari lagu dengan kata kunci judul, lalu ada logika while untuk mencari apakah ada lagu dengan judul yang sama dengan input kalau ada print dan set ditemukan true maka loop akan berhenti.


```

public static void main(String[] args)
{
    Musik_2511533018 playlist_3018 = new Musik_2511533018();
    Scanner input = new Scanner(System.in);
    int pilihan_3018;

    do {
        System.out.println("\n=== Playlist Musik NIM: 2511533018 ===");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan_3018 = input.nextInt();
        input.nextLine();

        switch (pilihan_3018) {
            case 1:
                System.out.print("Judul: ");
                String jdl = input.nextLine();
                System.out.print("Penyanyi: ");
                String pny = input.nextLine();
                playlist_3018.tambahLagu_3018(jdl, pny);
                break;
            case 2:
                playlist_3018.hapusLaguAwal_3018();
                break;
            case 3:
                playlist_3018.tampilMaju_3018();
                break;
            case 4:
                playlist_3018.tampilMundur_3018();
                break;
            case 5:
                System.out.print("Cari Judul: ");
                String cari = input.nextLine();
                playlist_3018.cariLagu_3018(cari);
                break;
            case 6:
                System.out.println("Program Selesai.");
                break;
            default:
                System.out.println("Pilihan tidak valid!");
        }
    } while (pilihan_3018 != 6);

    input.close();
}

```

Method main yang terdiri atas perulangan do berisi switch yang akan di eksekusi sesuai pilihan user.

Output:

=== Playlist Musik NIM: 2511533018 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 1

Judul: Lagu1

Penyanyi: penyanyi1

Lagu 'Lagu1' berhasil ditambahkan!

=== Playlist Musik NIM: 2511533018 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 1

Judul: lagu2

Penyanyi: penyanyi2

Lagu 'lagu2' berhasil ditambahkan!

=== Playlist Musik NIM: 2511533018 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 1

Judul: lagu3

Penyanyi: penyanyi3

Lagu 'lagu3' berhasil ditambahkan!

=== Playlist Musik NIM: 2511533018 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan:

--- Playlist (Maju) ---

- Lagu1 (penyanyi1)
- lagu2 (penyanyi2)
- lagu3 (penyanyi3)

--- Playlist (Mundur) ---

- lagu3 (penyanyi3)
- lagu2 (penyanyi2)
- Lagu1 (penyanyi1)

```
Menghapus lagu: Lagu1
```

```
=== Playlist Musik NIM: 2511533018 ===
```

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

```
Pilihan: 5
```

```
Cari Judul: lagu2
```

```
Lagu Ditemukan: lagu2 oleh penyany2
```

```
=== Playlist Musik NIM: 2511533018 ===
```

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

```
Pilihan: 2
```

```
Menghapus lagu: Lagu1
```

3.1 Ringkasan Hasil Laprak

Inti pembelajaran adalah bagaimana kita dapat memahami bagaimana Double Linked liST (DLL) dapat bekerja serta bagaimana implementasi langsungnya, dengan menggunakan banyak pendekatan, mulai dari pembuatan node, penelusuran, penghapusan, hingga menampilkan hasil program itu sendiri.